

```

import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.preprocessing import StandardScaler

# =====
# تحليل المعلمات Forward Pass التمرين 1: التحقق البرمجي من
# =====
print("="*60)
print("تحليل المعلمات Forward Pass : التمرين 1")
print("="*60)

# 1. بناء هيكل الشبكة العصبية
model_ex1 = Sequential([
    Dense(2, activation='relu', input_shape=(3,), name='Dense_Layer_1'),
    Dense(1, activation='sigmoid', name='Dense_Layer_2')
])

# 2. إدخال الأوزان والانحيازات المعطاة يدوياً في الامتحان
W1 = np.array([[0.2, 0.5],
               [0.3, 0.1],
               [0.4, 0.2]])
b1 = np.array([0.1, 0.2])

W2 = np.array([[0.7],
               [0.3]])
b2 = np.array([0.1])

# شحن الأوزان في الطبقات المناسبة
model_ex1.get_layer('Dense_Layer_1').set_weights([W1, b1])
model_ex1.get_layer('Dense_Layer_2').set_weights([W2, b2])

# 3. تمرير المدخلات وحساب المخرج النهائي (السؤال 1.1)
X_in = np.array([[0.5, 1.0, 2.0]])
y_pred_ex1 = model_ex1.predict(X_in)

print(f"\nالسؤال 1.1 المخرج النهائي المحسوب عبر الشبكة: {y_pred_ex1[0][0]:.4f}")

# 4. عرض المعلمات الإجمالية للطبقات للتحقق (السؤال 1.2)
print("\nالسؤال 1.2 ملخص الشبكة وحساب المعلمات القابلة للتدريب:")
model_ex1.summary()

```

```

# =====
# السؤال 1.3: التفسير النظري والبرمجي
# =====
print("\n" + "="*60)
print("السؤال 1.3: التفسير")
print("="*60)
predicted_class = 1 if y_pred_ex1[0][0] >= 0.5 else 0
print(f"بافتراض عتبة قرار مقدرة بـ 0.5:")
print(f"لأن القيمة ({predicted_class}) الفئة المتنبأ بها هي: الفئة -")
print(f"أكبر من 0.5 ({y_pred_ex1[0][0]:.4f}) الاحتمالية")
print(f"الآلية التي تسمح للشبكة بتعديل الأوزان عند الخطأ هي: الانتشار - (Backpropagation).")

# =====
# للتنبؤ بالاستهلاك الكهربائي لولاية البليدة Keras التمرين 2: بناء نموذج
# =====
print("\n" + "="*60)
print("للتنبؤ بالاستهلاك الكهربائي Keras التمرين 2: بناء نموذج")
print("="*60)

# تفريغ البيانات التاريخية المتوفرة في جدول الامتحان 1.
data = {
    'Temp': [8, 10, 14, 18, 24, 32, 38, 36, 28, 20, 12, 7],
    'Foyers': [1200, 1200, 1250, 1250, 1300, 1300, 1350, 1350, 1300, 1300, 1250, 1250],
    'Mois_num': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
    'Tarif': [4.5, 4.5, 4.5, 4.5, 4.5, 5.2, 5.2, 5.2, 4.5, 4.5, 4.5, 4.5],
    'Conso': [580, 520, 430, 350, 310, 480, 620, 590, 380, 330, 490, 600]
}

df = pd.DataFrame(data)

# (y) والمتغير التابع المراد التنبؤ به (X) فصل المتغيرات المستقلة
X_train = df[['Temp', 'Foyers', 'Mois_num', 'Tarif']].values
y_train = df['Conso'].values

# (StandardScaler) [الخطوة 1] تحضير البيانات بالتطبيع القياسي المستند إلى
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# [الخطوة 2] بناء نموذج تسلسلي من 3 طبقات كما هو محدد في نص التمرين
model_electricity = Sequential([
    Dense(16, activation='relu', input_shape=(4,), name='Hidden_Layer_1'),
    Dense(8, activation='relu', name='Hidden_Layer_2'),

```


Layer (type)	Output Shape	Param #
Dense_Layer_1 (Dense)	(None, 2)	8
Dense_Layer_2 (Dense)	(None, 1)	3

Total params: 11 (44.00 B)
Trainable params: 11 (44.00 B)
Non-trainable params: 0 (0.00 B)

السؤال 1.3: التفسير

بافتراض عتبة قرار مقدرة بـ 0.5:
- الفئة المتنبأ بها هي: الفئة (1) لأن القيمة الاحتمالية (0.7850) أكبر من 0.5
- الآلية التي تسمح للشبكة بتعديل الأوزان عند الخطأ هي: الانتشار العكسي للأخطاء (Backpropagation).

للتنبؤ بالاستهلاك الكهربائي Keras التمرين 2: بناء نموذج

...بدء تدريب نموذج الاستهلاك الكهربائي
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
تمت عملية التدريب بنجاح على 100 حقبة
1/1 ————— 0s 81ms/step
1/1 ————— 0s 44ms/step

النتائج التنبؤية للسيناريوهات المطلوبة:
MWh (فصل الصيف): A 0.49 التنبؤ بالاستهلاك للسيناريو
MWh (فصل الشتاء): B 0.56 التنبؤ بالاستهلاك للسيناريو

[السؤال 2.2] ملخص بنية نموذج الاستهلاك الكهربائي وحساب معلماته
Model: "sequential_3"

Layer (type)	Output Shape	Param #
Hidden_Layer_1 (Dense)	(None, 16)	80
Hidden_Layer_2 (Dense)	(None, 8)	136
Output_Layer (Dense)	(None, 1)	9

Total params: 677 (2.65 KB)
Trainable params: 225 (900.00 B)
Non-trainable params: 0 (0.00 B)
Optimizer params: 452 (1.77 KB)